

Used Car Price Prediction

Devixo Solutions — AI/ML Internship Task 04
End-to-End Machine Learning Application with Deployment

Project Type	End-to-End Regression ML Application
Dataset	Craigslist / Used Vehicles Dataset
Target Variable	price
Primary Model	Tuned Random Forest Regression Pipeline
Deployment	Streamlit
Model Serialization	Joblib

Prepared as an internship project report for Devixo Solutions.

1. Project Overview

The objective of this project is to build a complete machine learning application that accepts real-world used-vehicle information, processes the input through a trained regression pipeline, predicts the vehicle price, and presents the result through a Streamlit web application.

The workflow covers data preparation, feature engineering, model training, evaluation, hyperparameter tuning, model export, and deployment.

2. Dataset Description

The project uses a Craigslist used-vehicles dataset containing vehicle listings and associated prices and attributes.

Item	Value
Original records	426,880
Columns	26
Target	price
Invalid prices removed	32,895 records with price ≤ 0
Records after invalid-price removal	393,985
Duplicate records removed	0

3. Data Preprocessing

- Missing numerical values are handled using median imputation, while categorical values use most-frequent imputation.
- Categorical variables are encoded with OneHotEncoder using `handle_unknown='ignore'`. Numerical features are scaled with StandardScaler.
- Invalid target prices (price ≤ 0) were removed. Duplicate rows were checked; no duplicates were removed.
- A `vehicle_age` feature was engineered from the manufacturing year and a reference year.

4. Outlier Handling

The IQR method was used to identify unusually high prices. The calculated bounds were approximately $-23,885$ and $58,475$. Since non-positive prices had already been removed, the practical filtering step removes prices above the upper bound.

5. Train/Test Split

The cleaned dataset was split using an 80/20 train-test split with `random_state=42`.

Set	Records
Training	315,188
Testing	78,797

6. Machine Learning Models

Four regression algorithms are included in the comparison:

- Linear Regression — baseline regression model.
- Decision Tree — captures nonlinear relationships.
- Random Forest — ensemble of decision trees.
- Gradient Boosting — sequential boosting regression model.

7. Model Evaluation

Models are evaluated using MAE, MSE, RMSE, and R^2 . The final numerical values should be populated from results.csv and comparison.csv after the completed notebook run; no metric values are invented in this report.

Model	MAE	MSE	RMSE	R^2
Linear Regression	From results.csv	From results.csv	From results.csv	From results.csv
Decision Tree	From results.csv	From results.csv	From results.csv	From results.csv
Random Forest	From results.csv	From results.csv	From results.csv	From results.csv
Gradient Boosting	From results.csv	From results.csv	From results.csv	From results.csv
Tuned Random Forest	From comparison.csv	From comparison.csv	From comparison.csv	From comparison.csv

8. Hyperparameter Tuning

Random Forest was tuned because it is the main ensemble model used for the final application. The original GridSearchCV configuration creates 8 parameter combinations and 3 cross-validation folds, resulting in 24 model fits. For practical execution on the large dataset, the faster workflow uses RandomizedSearchCV with a 25,000-row tuning subset, 2-fold cross-validation, and 3 sampled combinations.

The tuned pipeline is obtained with:

```
best_model = tuning.best_estimator_
```

This object is already the complete preprocessing + Random Forest Pipeline and should not be followed by best_model.best_estimator_.

9. Feature Importance

Feature importance is extracted from the fitted Random Forest after obtaining transformed feature names from the preprocessing pipeline. The resulting feature_importance.csv is displayed by Streamlit as a ranked chart and table.

10. Model Export and Streamlit Deployment

The final pipeline is saved with Joblib. Streamlit loads the saved pipeline and uses model.predict() on user-entered vehicle information.

Artifact	Purpose
models/used_car_price_model.joblib	Saved final ML pipeline
artifacts/results.csv	Four-model performance comparison
artifacts/comparison.csv	Before vs after tuning metrics
artifacts/feature_importance.csv	Random Forest feature importance

Artifact	Purpose
artifacts/reference_year.csv	Reference year for vehicle-age calculation

Streamlit pages include Home / Prediction, Model Performance, Feature Importance, and Data Visualization.

11. Application Structure

Recommended deployment structure:

```
app.py
vehicles.csv
models/used_car_price_model.joblib
artifacts/results.csv
artifacts/comparison.csv
artifacts/feature_importance.csv
artifacts/reference_year.csv
```

12. Conclusion

This project implements an end-to-end machine learning workflow for used-car price prediction, including data cleaning, missing-value treatment, encoding, scaling, feature engineering, outlier handling, regression model comparison, Random Forest tuning, evaluation, serialization, and Streamlit deployment.

The final application accepts vehicle attributes and produces an estimated price through the same trained preprocessing and model pipeline. Exported artifacts support model-performance and feature-importance views in the Streamlit dashboard.

Note: final metric values should be inserted from the completed notebook artifacts after the final training/tuning run.